

Knapsack

```
#include<stdio.h>

int max(int a, int b)
{
    if (a>b)
        return a;
    else
        return b;
}

int main()
{
    int dp[20][20],wt[20],val[20];
    int i,j,n,w;

    printf("Enter the number of item:");
    scanf("%d",&n);
    printf("Enter the weight:");
    for(i=1;i<=n;i++)
        scanf("%d",&wt[i]);
    printf("Enter the value:");
    for(j=1;j<=n;j++)
        scanf("%d",&val[j]);
    printf("Enter the capacity:");
    scanf("%d",&w);

    for(i=0;i<=n;i++)
    {
        for(j=0;j<=w;j++)
        {
            if (i==0 || j==0)
                dp[i][j]=0;
            else if (wt[i]<=j)
                dp[i][j]=max(dp[i-1][j],val[i]+dp[i-1][j-
wt[i]]);
            else
                dp[i][j]=dp[i-1][j];
        }
    }
    printf("Max capacity:%d",dp[n][w]);
    return 0;
}
```

Subset sum

```
#include<stdio.h>

int i, j, n, k, target;
int set[20], subset[20];

void subsetsum(int i, int k, int sum)
{
    if(sum == target)
    {
        printf("Subset: ");
        for(j = 0; j < k; j++)
            printf("{ %d } ", subset[j]);
        printf("\n");
        return;
    }

    if(i == n || sum > target)
        return;

    subset[k] = set[i];
    subsetsum(i + 1, k + 1, sum + set[i]);
    subsetsum(i + 1, k, sum);
}

int main()
{
    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter the elements: ");
    for(i = 0; i < n; i++)
        scanf("%d", &set[i]);

    printf("Enter the target sum: ");
    scanf("%d", &target);

    subsetsum(0, 0, 0);

    return 0;
}
```

```
}
```

Floyd

```
#include<stdio.h>
#define max 100
#define inf 9999

int main()
{
    int i,j,k,n;
    int path[max][max];

    printf("Enter the number of vertex:");
    scanf("%d",&n);
    printf("Enter the adjancey matrix:\n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            scanf("%d",&path[i][j]);
        }
    }
    for(k=0;k<n;k++)
    {
        for(i=0;i<n;i++)
        {
            for(j=0;j<n;j++)
            {
                if (path[i][k]!=inf && path[k][j]!=inf &&
                    path[i][k]+path[k][j] < path[i][j])
                {
                    path[i][j] = path[i][k] + path[k][j];
                }
            }
        }
    }
    printf("shortest path:\n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            if (path[i][j]==inf)
```

```

        {
            printf("inf");
        }
        else
        {
            printf("%d ",path[i][j]);
        }
    }
    printf("\n");

}
return 0;
}

```

Warshall

```

#include<stdio.h>
#define max 100

int main()
{
    int i,j,k,n;
    int path[max][max];

    printf("Enter the number of vertex:");
    scanf("%d",&n);
    printf("Enter the adjancey matrix:\n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            scanf("%d",&path[i][j]);
        }
    }
    for(k=0;k<n;k++)
    {
        for(i=0;i<n;i++)
        {
            for(j=0;j<n;j++)
            {
                path[i][j]=path[i][j] || (path[i][k] && path[k][j]);
            }
        }
    }
}

```

```

    }
}
}
printf("Transitive matrix:\n");
for(i=0;i<n;i++)
{
    for(j=0;j<n;j++)
    {
        printf("%d ",path[i][j]);
    }
    printf("\n");
}
return 0;
}

```

Topological

```

#include<stdio.h>
#define max 100

int main()
{
    int i,j,n,found,count=0;
    int visited[max]={0};
    int graph[max][max];
    int indeegre[max]={0};

    printf("Enter the number of vertex:");
    scanf("%d",&n);

    printf("Enter the adjancey matrix:\n");
    for (i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            scanf("%d",&graph[i][j]);

        }
    }
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)

```

```

    {
        if(graph[i][j]==1)
        {
            indeegre[j]++;
        }
    }
}
printf("\n Topological order:\n");
while (count<n)
{
    found=0;
    for (i=0;i<n;i++)
    {
        if (indeegre[i]==0 && !visited[i])
        {
            printf("%d ",i);
            visited[i]=1;
            count++;
            found=1;

            for (j=0;j<n;j++)

            {
                if (graph[i][j]==1)
                {
                    indeegre[j]--;
                }
            }

        }
    }
    if (!found)
    {
        printf("Graph is not conncted");
        return 1;
    }
}
}

```

Prims

```
#include<stdio.h>
#define max 1000
#define inf 9999

int main()
{
    int i,j,a,n,b,min;
    int visited[max]={0};
    int cost[max][max];
    int total_cost=0,edges=0;

    printf("Enter the number of vertex:");
    scanf("%d",&n);
    printf("Enter the adjancey matrix:\n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            scanf("%d",&cost[i][j]);
            if (cost[i][j]==0)
                cost[i][j]=inf;

        }
    }
    visited[0]=1;
    printf("Edges in MST:");
    while(edges < n-1)
    {
        min = inf;

        for (i=0; i<n;i++)
        {
            if (visited[i])
            {
                for(j=0;j<n;j++)
                {
                    if (!visited[j] && cost[i][j] < min)
                    {
                        min = cost[i][j];
                        a=i;
                        b=j;
                    }
                }
            }
        }
    }
}
```

```

        }
    }

}

if (min == inf)
{
    printf("Graph is not connected");

}
printf("%d-->%d=%d",a,b,min);
total_cost+=min;
visited[b]=1;
edges++;
}
printf("Minimum Cost=%d",total_cost);

}

```

Merge

```

#include<stdio.h>
#include<stdlib.h>
#include<time.h>
void merge(int a[],int temp[],int low, int mid, int high)
{
    int i=low;
    int j=mid+1;
    int k=low;
    while(i <= mid && j<=high)
    {
        if ( a[i] <= a[j] )
        {
            temp[k++]=a[i++];

        }
        else
        {
            temp[k++]=a[j++];

        }
    }
    while (i<=mid)

```



```

        {
            temp[k++]=a[i++];

        }
        while (j <= high)
        {
            temp[k++]=a[j++];
        }
        for(i=low;i<=high;i++)
        {
            a[i] = temp[k];
        }

    }
}

void mergeSort(int a[],int temp[],int low , int high)
{
    if (low<high)
    {
        int mid = (low+high)/2;
        mergeSort(a,temp,low,mid);
        mergeSort(a,temp,mid+1,high);
        merge(a,temp,low,mid,high);
    }
}

int main()
{
    int *a;
    time_t start, end;
    double time_taken;
    int i,n;
    printf(("Enter the number of elemets is greater then 5000:"));
    scanf("%d",&n);
    a = (int *) malloc (n * sizeof (int));
    if (a==NULL)
    {
        printf("Memory not allocated");

    }

    srand(time(0));
    for (i=0;i<n;i++)
    {
        a[i] = rand()%10000;
    }
}

```

```

start = clock();
int *temp = (int *) malloc (n * sizeof (int));
mergeSort(a,temp,0,n-1);
end = clock();
time_taken = (double)(end-start)/CLOCKS_PER_SEC;
printf("Time taken to sort %d elemnts in %f seconds",n,time_taken);
free(temp);
return 0;
}

```

Quick

```

#include<stdio.h>
#include<stdlib.h>
#include<time.h>
int partition(int a[],int low, int high)
{
    int pivot=a[low];
    int i=low+1;
    int j=high;
    int temp;
    while(1)
    {
        while(i<=high && a[i]<=pivot)
        {
            i++;
        }
        while(a[j] > pivot)
        {
            j--;
        }
        if (i < j)
        {
            temp=a[i];
            a[i]=a[j];
            a[j]=temp;
        }
        else
        {
            temp=a[low];
            a[low]=a[j];
            a[j]=temp;
        }
    }
}

```

```

        return j;
    }
}

void quicksort(int a[],int low,int high)
{
    if (low < high)
    {
        int p=partition (a,low,high);
        quicksort (a,low,p-1);
        quicksort (a,p+1,high);
    }
}

int main()
{
    int *a;
    int n,i;
    clock_t start,end;
    double time_taken;
    printf("Enter the number of elements is greater then 5000:");
    scanf("%d",&n);
    a=(int *)malloc (n* sizeof(int));
    if (a==0)
    {
        printf("memory not allocated");
        return 0;
    }

    srand(time(0));
    for(i=0;i<n;i++)
    {
        a[i]=rand()%1000;
    }
    start=clock();
    quicksort(a,0,n-1);
    end=clock();
    time_taken=(double)(end-start)/CLOCKS_PER_SEC;
    printf("The time taken to sort %d elements is %f
seconds\n",n,time_taken);

}

```

Selection

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
int i,j,n,min,temp;
void selection_sort(int arr[],int n)
{
    for(i=0;i<n-1;i++)
    {
        min=i;
        for(j=i+1;j<n;j++)
        {
            if (arr[j]<arr[min])
            {
                min=j;
            }
        }
        temp=arr[i];
        arr[i]=arr[min];
        arr[min]=temp;
    }
}
int main()
{
    int *arr;
    clock_t start,end;
    double time_taken;
    printf("Enter the number of elements is greater than 5000:");
    scanf("%d",&n);
    arr=(int*)malloc(n*sizeof(int));

    if (arr==0)
    {
        printf("Memory not allocated");
    }
    srand(time(0));
    for(i=0;i<n;i++)
    {
```

```

        arr[i]=rand()%1000;
        start=clock();
        selection_sort(arr,n);
        end=clock();
        time_taken=((double)(end-start))/CLOCKS_PER_SEC;
        printf("Time taken for sorting %d elements is %f
seconds\n",n,time_taken);
        free(arr);
        return 0;
    }
}

```

Dijkstra's algorithm.

```

#include<stdio.h>

#define MAX 20
#define INF 9999

int main()
{
    int cost[MAX][MAX], dist[MAX], visited[MAX]={0};
    int i, j, n, s, min, u;

    printf("Enter number of vertices: ");
    scanf("%d",&n);

    printf("Enter cost matrix:\n");
    for(i=0;i<n;i++)
        for(j=0;j<n;j++)
            scanf("%d",&cost[i][j]);

    printf("Enter source: ");
    scanf("%d",&s);

    for(i=0;i<n;i++)
        dist[i]=cost[s][i];

    dist[s]=0;
    visited[s]=1;

    for(i=1;i<n;i++)

```

```

{
    min=INF;

    for(j=0;j<n;j++)
        if(!visited[j] && dist[j]<min)
        {
            min=dist[j];
            u=j;
        }

    visited[u]=1;

    for(j=0;j<n;j++)
        if(!visited[j] && dist[u]+cost[u][j]<dist[j])
            dist[j]=dist[u]+cost[u][j];
}

printf("Shortest Distances:\n");
for(i=0;i<n;i++)
    printf("%d -> %d = %d\n", s, i, dist[i]);

return 0;
}

```